

Chapter 11: Strings in Java

Personal Notes

Until now you have learned variables (single values), arrays (multiple values), loops (repeating logic), and conditionals (making decisions). Now comes a topic you will use in literally every real program you ever write — Strings.

If arrays taught you how to handle data, strings teach you how to handle text. Almost every interview question, every form input, every search bar, and every chat message involves strings. Getting comfortable here pays back forever.

1. What is a String in Java?

A String is a sequence of characters. In simpler words, a String is a collection of characters enclosed in double quotes.

```
String name = "Aman";  
String course = "Java DSA";  
String message = "Welcome to Java!";
```

Real-Life Examples

Strings show up in almost every part of every app you use:

- Usernames — "Riya"
- Course or product names — "DSA Bootcamp"
- Messages and notifications — "Welcome back!"
- Emails — "user@gmail.com"
- Passwords (handled as strings before being hashed)
- Search queries, comments, tags, file names — all strings

2. Why Strings Are Important

Strings are used everywhere in real programming:

- User input from forms or the keyboard
- Search systems and filters
- Text processing (cleaning, formatting, splitting)
- Validation (email format, password rules, phone numbers)
- Logs and error messages
- DSA problem-solving (palindromes, anagrams, pattern matching)

3. How to Create Strings

There are two ways to create a String in Java. Both look similar but behave slightly differently behind the scenes.

Method 1: Using a String Literal (most common)

```
String str = "Hello";
```

This is the way you should use 99% of the time. It is short, clean, and lets Java reuse memory automatically.

Method 2: Using the new keyword

```
String str = new String("Hello");
```

This forces Java to create a new object in the heap every time, even if the same text already exists. It uses more memory and is rarely needed.

For beginners: Always use Method 1 (the literal). Method 2 only matters when you're learning about the String Pool and memory optimization.

4. String is NOT a Primitive Type

This is one of the most important things to understand about Strings. Unlike int, double, or char, String is NOT a primitive type.

A String is an object of the java.lang.String class. That is why you can call methods on it like:

- `str.length()` — get the number of characters
- `str.charAt(i)` — get the character at index `i`
- `str.equals(other)` — compare with another string
- `str.toUpperCase()` — get an UPPERCASE version

You cannot call methods on primitives like int — you can never write `5.something()`. But you can on Strings because they are objects.

5. String Basics

5.1 Length of a String

```
String str = "Programming";  
System.out.println(str.length()); // 11
```

Notice the parentheses! For Strings, `length` is a METHOD, so you write `length()`. For arrays, `length` is a PROPERTY, so you write `length` (no parentheses). This catches a lot of beginners.

5.2 Accessing Characters

```
String str = "Java";

System.out.println(str.charAt(0)); // J
System.out.println(str.charAt(1)); // a
System.out.println(str.charAt(3)); // a
```

Index Layout

Index:	0	1	2	3
	+-----+	+-----+	+-----+	+-----+
Char:	J	a	v	a
	+-----+	+-----+	+-----+	+-----+

Important rule: String indices start at 0, just like arrays. The last valid index is `length() - 1`. Trying to access `str.charAt(str.length())` throws `StringIndexOutOfBoundsException`.

6. Strings Are Immutable (VERY IMPORTANT)

This is the single most important rule of Strings in Java: once a String is created, it CANNOT be changed. Any method that looks like it changes the String actually creates a brand new String.

The classic trap

```
String str = "Hello";
str.concat(" World");

System.out.println(str); // Hello (NOT "Hello World!")
```

Beginners expect this to print "Hello World" but it just prints "Hello". Why? Because `concat()` does not modify `str`. It creates a new String "Hello World" — and we threw it away without saving it.

The correct way

```
String str = "Hello";
str = str.concat(" World"); // assign the new String back

System.out.println(str); // Hello World
```

Memory Picture

Before concat:	After concat (saved back):
str ---> "Hello"	str ---> "Hello World"
	"Hello" (still in memory, but nothing points to it → garbage collected later)

Why this design? Immutability makes Strings safer to share between methods and threads, and it allows Java to optimize memory through the String Pool. The trade-off is that beginners have to remember: always assign the result back.

7. Comparing Strings (VERY IMPORTANT)

This is one of the most asked Java interview questions and one of the most common bug sources for beginners. Read this section carefully.

7.1 The Wrong Way: ==

```
String a = new String("hello");
String b = new String("hello");

if (a == b) {
    System.out.println("Same");
} else {
    System.out.println("Different");
}
// Output: Different
```

Even though both strings contain "hello", == returns false. Why?

7.2 The Correct Way: .equals()

```
if (a.equals(b)) {
    System.out.println("Same");
} else {
    System.out.println("Different");
}
// Output: Same
```

7.3 What is actually happening

==	→ compares MEMORY ADDRESSES "are these the same object?"
.equals()	→ compares the actual CHARACTERS "do these have the same text?"

Two different String objects can hold the exact same text but live at different memory addresses. == sees them as different. equals() looks at the actual characters and says they are the same.

Rule of thumb: ALWAYS use .equals() to compare String content. Never use == unless you're specifically checking if two references point to the exact same object (which is rare).

Bonus: equalsIgnoreCase()

To compare while ignoring uppercase vs lowercase differences, use equalsIgnoreCase().

```
"Java".equalsIgnoreCase("JAVA");    // true
"Java".equals("JAVA");               // false
```

8. Taking String Input

Use the Scanner class to read strings from the user.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String name = sc.next();          // reads ONE word
        String fullName = sc.nextLine();  // reads the WHOLE line

        sc.close();
    }
}
```

Difference between next() and nextLine()

Method	Reads	Example
next()	Reads one word (stops at space)	"Aman Sharma" → reads only "Aman"
nextLine()	Reads the entire line	"Aman Sharma" → reads "Aman Sharma"

Gotcha: If you use nextInt() followed by nextLine(), the nextLine() will read an empty string. This is because nextInt() leaves a newline in the buffer. Add an extra sc.nextLine() between them to clear it.

9. Basic String Problems

Problem 1: Print Each Character

```
public class Main {
    public static void main(String[] args) {
        String str = "Code";

        for (int i = 0; i < str.length(); i++) {
            System.out.println(str.charAt(i));
        }
    }
}
```

Output:

```
C
o
d
e
```

Key learning: String traversal works exactly like array traversal — loop from 0 to length() - 1.

Problem 2: Count Length Without length()

This is just practice — in real code, always use length().

```
String str = "Java";
int count = 0;

for (int i = 0; i < str.length(); i++) {
    count++;
}

System.out.println(count);    // 4
```

Problem 3: Count Vowels

```
String str = "CodeHelp";
int count = 0;

for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);

    if (ch == 'a' || ch == 'e' || ch == 'i' ||
        ch == 'o' || ch == 'u' ||
        ch == 'A' || ch == 'E' || ch == 'I' ||
        ch == 'O' || ch == 'U') {
        count++;
    }
}

System.out.println("Vowels: " + count);
```

Dry Run for "CodeHelp"

```
C   o   d   e   H   e   l   p
  ^       ^       ^
  v       v       v
  o       e       e

Vowels found: o, e, e
Total = 3
```

Smarter version: You can also write `"aeiouAEIOU".indexOf(ch) != -1` — that one expression replaces the entire long OR chain.

Problem 4: Reverse a String

```
String str = "Java";
String reversed = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reversed = reversed + str.charAt(i);
}

System.out.println(reversed); // avaJ
```

Loop from the last index down to 0, and append each character to a new string. This is one of the most common interview warm-up questions.

Problem 5: Check Palindrome

A palindrome reads the same forward and backward — like madam, racecar, or level.

```
String str = "madam";
String reversed = "";

for (int i = str.length() - 1; i >= 0; i--) {
    reversed += str.charAt(i);
}

if (str.equals(reversed)) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
```

Output: Palindrome

Notice the use of `.equals()` instead of `==` — this is exactly the case where the difference matters.

10. Common String Methods

Method	Returns	What it does
<code>length()</code>	int	Returns number of characters
<code>charAt(i)</code>	char	Returns character at index i
<code>equals(s)</code>	boolean	True if two strings have same content
<code>equalsIgnoreCase(s)</code>	boolean	Same as equals but case-insensitive

Method	Returns	What it does
<code>toLowerCase()</code>	String	Returns lowercase version
<code>toUpperCase()</code>	String	Returns uppercase version
<code>concat(s)</code>	String	Joins two strings (same as +)
<code>indexOf(ch)</code>	int	Position of first occurrence (-1 if absent)
<code>substring(a, b)</code>	String	Characters from index a to b-1
<code>trim()</code>	String	Removes leading and trailing spaces
<code>replace(a, b)</code>	String	Replaces all 'a' with 'b'
<code>contains(s)</code>	boolean	True if string contains substring s
<code>split(regex)</code>	String[]	Splits string into an array
<code>isEmpty()</code>	boolean	True if length is 0

Examples

```
String str = "  Hello Java  ";

str.length();           // 14
str.charAt(2);           // 'H'
str.trim();              // "Hello Java"
str.toUpperCase();       // "  HELLO JAVA  "
str.indexOf('J');        // 8
str.substring(2, 7);     // "Hello"
str.replace('l', 'L');   // "  HeLLo Java  "
str.contains("Java");    // true
```

11. Common Mistakes

Mistake 1: Using == for comparison

```
if (str1 == str2) { ... } // WRONG – compares addresses
if (str1.equals(str2)) { ... } // correct
```

Mistake 2: Going out of bounds

```
String s = "Java";
s.charAt(4); // ERROR – last valid index is 3
```

Mistake 3: Forgetting immutability

```
str.concat(" World"); // WRONG – result is thrown away
```



```
str = str.concat(" World"); // correct – save the new value
```

Mistake 4: Confusing next() and nextLine()

next() stops at the first space, so "Aman Sharma" only reads "Aman". If you want full input including spaces, use nextLine().

Mistake 5: Using length vs length()

```
arr.length    // arrays    (no parentheses, it's a property)
str.length()  // strings    (parentheses, it's a method)
```

Mistake 6: Building strings inside a big loop

Every += on a String creates a new object. For thousands of iterations, this is slow. For heavy string building, use StringBuilder (you'll meet it later).

12. Homework / Practice Problems

Try these on your own. They cover the most common interview-style String questions.

1. Count consonants in a string (everything that is NOT a vowel).
2. Convert a string to uppercase WITHOUT using toUpperCase().
3. Find the frequency of a specific character in a string.
4. Remove all spaces from a string.
5. Check if a string contains only digits.
6. Count the number of words in a sentence.
7. Print the longest word in a sentence.
8. Check if two strings are anagrams of each other.
9. Reverse each word in a sentence ("Hello World" → "olleH dlroW").
10. Find the first non-repeating character in a string.

13. Interview Importance

Strings are one of the most-asked topics across coding rounds, DSA, and backend interviews. The concepts you build here lead to:

- Sliding window problems (longest substring without repeating characters)
- Hashing and frequency counting
- Pattern matching (KMP, Rabin-Karp)
- Substring and palindrome problems
- String manipulation in system design
- Regular expressions and input validation

14. Quick Recap

- Strings are sequences of characters wrapped in double quotes
- String is an object, not a primitive — so it has methods
- Use `length()` (with parentheses) and `charAt(i)` for basic operations
- Strings are immutable — every "change" creates a new String
- ALWAYS use `.equals()` for comparing, never `==`
- Use `next()` for single words, `nextLine()` for full lines
- Master common methods: `trim`, `substring`, `indexOf`, `replace`, `contains`

15. Final Takeaway

If arrays taught you how to handle data, strings teach you how to handle text data.

Almost every real-world problem involves strings somewhere — names, emails, logs, search queries, file paths. This is why building a strong foundation here matters so much. Master these basics, and the harder topics (substrings, sliding window, pattern matching, anagrams) become natural extensions.

Key Takeaway: Two rules will save you 90% of String bugs: (1) Strings are immutable — always assign the result back. (2) Use `.equals()` for comparison, never `==`. Internalize these and you'll write clean String code from day one.

16. What's Next?

Once Strings feel natural, the next logical steps are:

- `StringBuilder` — a faster, mutable version of String for heavy modification
- Character class methods (`Character.isDigit`, `Character.isLetter`, etc.)
- Regular expressions (regex) for advanced pattern matching
- Sliding window technique on strings
- Hashing and frequency maps for string problems
- Recursion with strings